

Data Structures- Lists, Tuples, Dicts, Sets



Suman

24 Jan, 2026 At 8:00 PM

Notes

Trimester-I

Recommended

Resource



Associated Lectures (1)



Description



Discussions

Session 2.1: Control Flow, Loops & Data Structures

AIM101 Python Fundamentals

During/After Class Reference | Duration: 105 minutes

In-Class Resources

Mental Map

```
SESSION 2.1: CONTROL FLOW, LOOPS & DATA STRUCTURES
|
+-- A. CONDITIONALS
|   +-- if / elif / else
|   +-- Comparison & Logical Operators
|   +-- Nested Conditionals
|
+-- B. LOOPS
|   +-- for loops with range()
|   +-- while loops
|   +-- break & continue
|   +-- Loop Patterns (Counter, Accumulator, Search)
|
+-- C. DATA STRUCTURES
|   +-- Lists (Mutable, Ordered)
|   +-- Tuples (Immutable, Ordered)
|   +-- Dictionaries (Key-Value Pairs)
|   +-- Sets (Unique, Unordered)
|
+-- D. MINI-PROJECT
    +-- Contact Book Application
```

The Big Picture

Why This Session Matters

Every useful program needs three fundamental capabilities:

Make Decisions - Should we approve this loan? Is this email spam? Programs must evaluate conditions and choose paths.

Repeat Actions - Process 10,000 customer records, retry a failed connection, keep asking until valid input is received. Loops eliminate tedious repetition.

Organize Data - Store customer contacts, track inventory counts, maintain unique visitor IDs. Different data structures excel at different tasks.

Real-World Connection: Consider a food delivery app:

- **Conditionals** decide if a restaurant is open, if the order meets minimum amount
- **Loops** iterate through menu items, calculate totals, process multiple orders
- **Data Structures** store the menu (dictionary), order items (list), delivery zones (set)

By the end of this session, you will build a **Contact Book** that combines all three capabilities.

Section A: if / elif / else Statements

The Concept

Conditionals allow programs to make decisions. Think of them as **digital traffic signals** - they direct the flow of your program based on whether conditions are true or false.

Syntax Structure

```
if condition:
    # Code block executes if condition is True
    statement1
    statement2
elif another_condition:
    # Code block executes if first condition is False AND this is True
    statement3
else:
    # Code block executes if ALL above conditions are False
    statement4
```

Comparison Operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True

Operator	Meaning	Example	Result
<code>!=</code>	Not equal to	<code>5 != 3</code>	True
<code><</code>	Less than	<code>3 < 5</code>	True
<code>></code>	Greater than	<code>5 > 3</code>	True
<code><=</code>	Less than or equal	<code>5 <= 5</code>	True
<code>>=</code>	Greater than or equal	<code>5 >= 3</code>	True

Logical Operators

Operator	Meaning	Example	Result
<code>and</code>	Both must be True	<code>True and False</code>	False
<code>or</code>	At least one True	<code>True or False</code>	True
<code>not</code>	Inverts the value	<code>not True</code>	False

Practical Examples

Example 1: Grade Classification

```
score = 78

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print(f"Score: {score}, Grade: {grade}")
# Output: Score: 78, Grade: C
```

Example 2: Loan Eligibility (Multiple Conditions)

```
age = 28
income = 55000
credit_score = 720

if age >= 21 and income >= 30000 and credit_score >= 650:
    print("Loan Approved")
else:
    print("Loan Denied")
# Output: Loan Approved
```

Example 3: Weekend or Weekday

```
day = "Saturday"

if day == "Saturday" or day == "Sunday":
    print("Weekend - Relax!")
else:
    print("Weekday - Work time!")
# Output: Weekend - Relax!
```

Key Rules

Colon Required - Every `if`, `elif`, `else` line must end with `:`

Indentation Matters - Code blocks must be indented (4 spaces standard)

Order Matters - Conditions are checked top-to-bottom; first True wins

elif is Optional - You can have `if-else` without `elif`

else is Optional - You can have `if` alone or `if-elif` without `else`

Section B: Nested Conditionals

The Concept

Nested conditionals are conditionals inside other conditionals. Think of them as **decision trees** where one decision leads to more decisions.

When to Use

- When decisions depend on previous decisions
- When you have hierarchical logic
- When combining would create complex single conditions

Syntax Structure

```
if outer_condition:
    if inner_condition:
        # Both outer AND inner are True
        statement1
    else:
        # Outer is True, inner is False
        statement2
else:
    # Outer is False
    statement3
```

Practical Example: ATM Withdrawal

```
has_card = True
correct_pin = True
balance = 5000
withdrawal_amount = 2000

if has_card:
    if correct_pin:
        if withdrawal_amount <= balance:
            balance = balance - withdrawal_amount
            print(f"Dispensing ${withdrawal_amount}")
            print(f"New balance: ${balance}")
        else:
            print("Insufficient funds")
    else:
        print("Incorrect PIN")
else:
    print("Please insert card")

# Output:
# Dispensing $2000
# New balance: $3000
```

Flattening Nested Conditionals

Sometimes nested conditionals can be simplified using logical operators:

```
# Nested version
if age >= 18:
    if has_license:
        print("Can drive")

# Flattened version (equivalent)
if age >= 18 and has_license:
    print("Can drive")
```

Rule of Thumb: If nesting exceeds 3 levels, consider refactoring.

Section C: for Loops & range()

The Concept

A `for` loop repeats code for each item in a sequence. Think of it as a **factory assembly line** - the same operation is performed on each item that comes through.

Syntax Structure

```
for variable in sequence:
    # Code block executes once for each item
```

```
statement1  
statement2
```

The range() Function

`range()` generates a sequence of numbers. It is the most common way to control loop iterations.

Syntax	Description	Output
<code>range(5)</code>	0 to 4 (5 numbers)	0, 1, 2, 3, 4
<code>range(2, 6)</code>	2 to 5	2, 3, 4, 5
<code>range(0, 10, 2)</code>	0 to 8, step 2	0, 2, 4, 6, 8
<code>range(5, 0, -1)</code>	5 to 1, countdown	5, 4, 3, 2, 1

Practical Examples

Example 1: Basic Counting

```
for i in range(5):  
    print(f"Count: {i}")
```

```
# Output:  
# Count: 0  
# Count: 1  
# Count: 2  
# Count: 3  
# Count: 4
```

Example 2: Iterating Over a List

```
fruits = ["apple", "banana", "cherry"]  
  
for fruit in fruits:  
    print(f"I like {fruit}")
```

```
# Output:  
# I like apple  
# I like banana  
# I like cherry
```

Example 3: Using Index with enumerate()

```
students = ["Alice", "Bob", "Charlie"]  
  
for index, name in enumerate(students):  
    print(f"Student {index + 1}: {name}")
```

```
# Output:  
# Student 1: Alice
```

```
# Student 2: Bob
# Student 3: Charlie
```

Example 4: Multiplication Table

```
number = 5

for i in range(1, 11):
    result = number * i
    print(f"{number} x {i} = {result}")

# Output:
# 5 x 1 = 5
# 5 x 2 = 10
# ... (continues to 5 x 10 = 50)
```

Key Points

- Loop variable is automatically updated each iteration
- `range()` is memory-efficient (generates numbers on demand)
- Loop variable exists after loop ends (contains last value)

Section D: while Loops

The Concept

A `while` loop repeats code as long as a condition remains True. Think of it as a **security guard** who keeps checking "Is the condition still true?" before allowing another round.

Syntax Structure

```
while condition:
    # Code block executes while condition is True
    statement1
    statement2
    # Usually includes something that changes the condition
```

Practical Examples

Example 1: Countdown

```
count = 5

while count > 0:
    print(count)
    count = count - 1 # CRITICAL: Must change condition

print("Liftoff!")
```

```
# Output:  
# 5  
# 4  
# 3  
# 2  
# 1  
# Liftoff!
```

Example 2: Input Validation

```
password = ""  
  
while password != "secret123":  
    password = input("Enter password: ")  
    if password != "secret123":  
        print("Incorrect, try again.")  
  
print("Access granted!")
```

Example 3: Unknown Iteration Count

```
import random  
  
attempts = 0  
target = random.randint(1, 10)  
guess = 0  
  
while guess != target:  
    guess = int(input("Guess (1-10): "))  
    attempts += 1  
    if guess < target:  
        print("Too low!")  
    elif guess > target:  
        print("Too high!")  
  
print(f"Correct! It took {attempts} attempts.")
```

Infinite Loop Warning

```
# DANGER: Infinite loop – condition never becomes False  
x = 1  
while x > 0:  
    print(x)  
    x = x + 1 # x keeps increasing, never <= 0
```

Prevention: Always ensure something in the loop body can make the condition False.

Section E: break, continue & Loop Patterns

break Statement

`break` immediately exits the loop, skipping any remaining iterations.

```
# Find first negative number
numbers = [4, 7, -2, 9, 1]

for num in numbers:
    if num < 0:
        print(f"Found negative: {num}")
        break
    print(f"Checking: {num}")
```

```
# Output:
# Checking: 4
# Checking: 7
# Found negative: -2
```

continue Statement

`continue` skips the rest of the current iteration and moves to the next.

```
# Print only positive numbers
numbers = [3, -1, 4, -5, 2]

for num in numbers:
    if num < 0:
        continue # Skip negative numbers
    print(num)
```

```
# Output:
# 3
# 4
# 2
```

Common Loop Patterns

Pattern 1: Counter Pattern

```
# Count occurrences
text = "hello world"
vowel_count = 0

for char in text:
    if char in "aeiou":
        vowel_count += 1

print(f"Vowels: {vowel_count}") # Output: Vowels: 3
```

Pattern 2: Accumulator Pattern

```
# Calculate sum
numbers = [10, 20, 30, 40]
total = 0 # Accumulator starts at 0

for num in numbers:
    total += num

print(f"Sum: {total}") # Output: Sum: 100
```

Pattern 3: Search Pattern

```
# Find specific item
names = ["Alice", "Bob", "Charlie", "Diana"]
search_for = "Charlie"
found = False

for name in names:
    if name == search_for:
        found = True
        break

if found:
    print(f"{search_for} is in the list")
else:
    print(f"{search_for} not found")
```

Pattern 4: Filter Pattern

```
# Collect items meeting criteria
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = []

for num in numbers:
    if num % 2 == 0:
        evens.append(num)

print(f"Even numbers: {evens}") # Output: Even numbers: [2, 4, 6, 8, 10]
```

Section F: Lists

The Concept

A list is an **ordered, mutable collection** of items. Think of it as a **numbered locker system** - each item has a position (index), and you can change what is in any locker.

Creating Lists

```
# Empty list
empty = []
```

```
# List with values
fruits = ["apple", "banana", "cherry"]

# Mixed types (allowed but uncommon)
mixed = [1, "hello", 3.14, True]

# List from range
numbers = list(range(1, 6)) # [1, 2, 3, 4, 5]
```

Indexing

```
fruits = ["apple", "banana", "cherry", "date"]

# Positive indexing (from start)
print(fruits[0]) # apple (first item)
print(fruits[2]) # cherry (third item)

# Negative indexing (from end)
print(fruits[-1]) # date (last item)
print(fruits[-2]) # cherry (second to last)
```

Visual Index Map:

	+-----+	+-----+	+-----+	+-----+
Elements:	apple	banana	cherry	date
	+-----+	+-----+	+-----+	+-----+
Positive:	0	1	2	3
Negative:	-4	-3	-2	-1

Slicing

```
fruits = ["apple", "banana", "cherry", "date", "elderberry"]

# Basic slicing: list[start:end] (end is exclusive)
print(fruits[1:4]) # ['banana', 'cherry', 'date']
print(fruits[:3]) # ['apple', 'banana', 'cherry']
print(fruits[2:]) # ['cherry', 'date', 'elderberry']

# With step: list[start:end:step]
print(fruits[::2]) # ['apple', 'cherry', 'elderberry']
print(fruits[::-1]) # Reversed list
```

Essential List Methods

Method	Description	Example
<code>append(x)</code>	Add item to end	<code>fruits.append("fig")</code>
<code>insert(i, x)</code>	Insert at index i	<code>fruits.insert(0, "apricot")</code>

Method	Description	Example
<code>remove(x)</code>	Remove first occurrence	<code>fruits.remove("banana")</code>
<code>pop(i)</code>	Remove & return item at index	<code>fruits.pop(1)</code>
<code>pop()</code>	Remove & return last item	<code>fruits.pop()</code>
<code>index(x)</code>	Find index of item	<code>fruits.index("cherry")</code>
<code>count(x)</code>	Count occurrences	<code>fruits.count("apple")</code>
<code>sort()</code>	Sort in place	<code>fruits.sort()</code>
<code>reverse()</code>	Reverse in place	<code>fruits.reverse()</code>
<code>len(list)</code>	Get length	<code>len(fruits)</code>

Practical Example: Shopping Cart

```

cart = []

# Add items
cart.append("Laptop")
cart.append("Mouse")
cart.append("Keyboard")
print(f"Cart: {cart}") # ['Laptop', 'Mouse', 'Keyboard']

# Insert at position
cart.insert(1, "Monitor")
print(f"Cart: {cart}") # ['Laptop', 'Monitor', 'Mouse', 'Keyboard']

# Remove item
cart.remove("Mouse")
print(f"Cart: {cart}") # ['Laptop', 'Monitor', 'Keyboard']

# Check item count
print(f"Items in cart: {len(cart)}") # 3

```

Section G: Tuples & Immutability

The Concept

A tuple is an **ordered, immutable collection**. Think of it as a **sealed envelope** - once sealed, the contents cannot be changed.

Creating Tuples

```

# With parentheses
coordinates = (10, 20)

```

```
# Without parentheses (tuple packing)
point = 3, 4, 5

# Single element tuple (comma required!)
single = (42,) # This is a tuple
not_tuple = (42) # This is just an integer!

# Empty tuple
empty = ()
```

Accessing Tuple Elements

```
person = ("Alice", 28, "Engineer")

# Indexing (same as lists)
print(person[0]) # Alice
print(person[-1]) # Engineer

# Slicing
print(person[1:]) # (28, 'Engineer')

# Unpacking
name, age, job = person
print(f"{name} is {age} years old") # Alice is 28 years old
```

Why Immutability Matters

```
point = (10, 20)

# This will raise an error
# point[0] = 15 # TypeError: 'tuple' object does not support item assignment

# To "change" a tuple, create a new one
new_point = (15, point[1])
print(new_point) # (15, 20)
```

Tuples vs Lists: When to Use Each

Use Tuples When	Use Lists When
Data should not change	Data needs modification
Representing fixed records	Building collections dynamically
Dictionary keys needed	Order-based operations
Function returning multiple values	Stack/queue operations

Practical Example: Returning Multiple Values

```
def get_min_max(numbers):  
    return (min(numbers), max(numbers))  
  
data = [45, 22, 89, 12, 67]  
minimum, maximum = get_min_max(data)  
print(f"Min: {minimum}, Max: {maximum}") # Min: 12, Max: 89
```

Section H: Sets & Data Structure Selection

The Concept

A set is an **unordered collection of unique elements**. Think of it as a **bag of unique tokens** - no duplicates allowed, and you cannot pick by position.

Creating Sets

```
# Using curly braces  
fruits = {"apple", "banana", "cherry"}  
  
# From list (removes duplicates)  
numbers = [1, 2, 2, 3, 3, 3]  
unique = set(numbers)  
print(unique) # {1, 2, 3}  
  
# Empty set (NOT {} - that's an empty dict!)  
empty = set()
```

Set Operations

```
a = {1, 2, 3, 4}  
b = {3, 4, 5, 6}  
  
# Union (items in either set)  
print(a | b) # {1, 2, 3, 4, 5, 6}  
print(a.union(b)) # Same result  
  
# Intersection (items in both sets)  
print(a & b) # {3, 4}  
print(a.intersection(b)) # Same result  
  
# Difference (items in a but not b)  
print(a - b) # {1, 2}  
print(a.difference(b)) # Same result  
  
# Symmetric difference (items in one but not both)  
print(a ^ b) # {1, 2, 5, 6}  
print(a.symmetric_difference(b)) # Same result
```

Set Methods

```
colors = {"red", "green", "blue"}

# Add element
colors.add("yellow")

# Remove element (raises error if not found)
colors.remove("red")

# Discard element (no error if not found)
colors.discard("purple") # No error

# Check membership (very fast!)
print("green" in colors) # True
```

Data Structure Selection Guide

Need	Best Choice	Why
Ordered collection, may change	List	Supports indexing, mutable
Fixed record of values	Tuple	Immutable, can be dict key
Key-value lookups	Dictionary	O(1) access by key
Unique items, no order needed	Set	Auto-removes duplicates, fast membership
Stack (LIFO)	List	append() and pop()
Queue (FIFO)	collections.deque	Efficient both ends

Practical Example: Removing Duplicates

```
# Problem: Find unique visitors
visitors = ["alice", "bob", "alice", "charlie", "bob", "diana"]

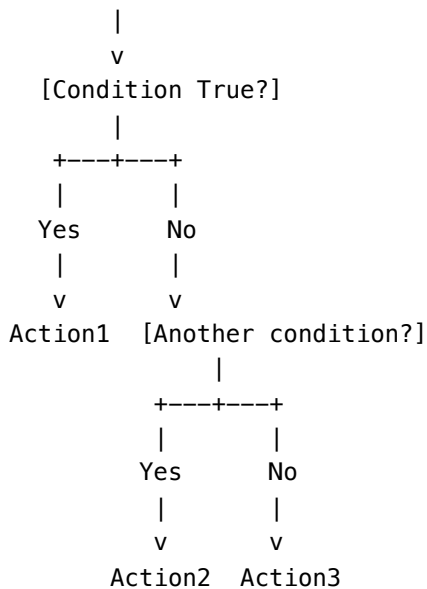
# Solution: Convert to set
unique_visitors = set(visitors)
print(f"Unique visitors: {unique_visitors}")
# Output: Unique visitors: {'alice', 'bob', 'charlie', 'diana'}

print(f"Total unique: {len(unique_visitors)}") # 4
```

Key Frameworks

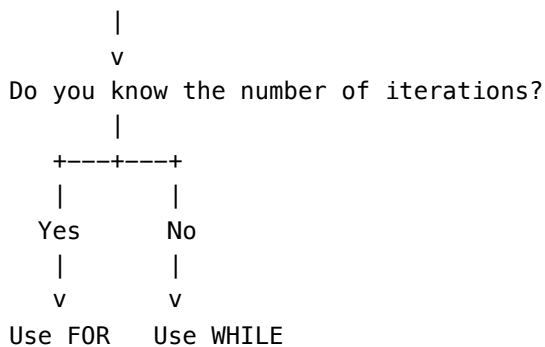
Decision Tree Pattern

Start: What condition to check?



Loop Selection Guide

Need to iterate?



Use **for** when:

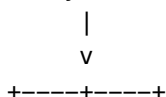
- Iterating over a sequence (list, string, range)
- You know how many times to loop
- Processing each item in a collection

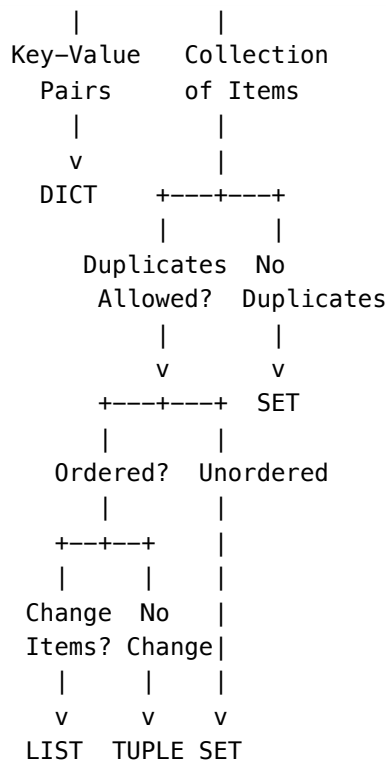
Use **while** when:

- Waiting for a condition to change
- Unknown number of iterations
- User input validation
- Running until stopped

Data Structure Decision Guide

What do you need to store?





Common Errors Quick Reference

Error	Cause	Fix
<code>IndentationError</code>	Inconsistent spaces/tabs	Use 4 spaces consistently
<code>SyntaxError: invalid syntax</code>	Missing colon after if/for/while	Add <code>:</code> at end of line
<code>IndexError: list index out of range</code>	Accessing index that does not exist	Check <code>len()</code> before accessing
<code>KeyError</code>	Dictionary key does not exist	Use <code>get()</code> or check <code>in</code> first
<code>TypeError: 'tuple' object does not support item assignment</code>	Trying to modify tuple	Use list instead, or create new tuple
<code>TypeError: unhashable type: 'list'</code>	Using list as dict key or set element	Use tuple instead
<code>NameError</code>	Variable used before assignment	Initialize variable before loop
Infinite loop	While condition never becomes False	Ensure loop body changes condition

Key Takeaways

Conditionals direct program flow - `if` checks once, `elif` provides alternatives, `else` is the default

Comparison operators (`==` , `!=` , `<` , `>` , `<=` , `>=`) produce Boolean results; **logical operators** (`and` , `or` , `not`) combine them

for loops iterate a known number of times; **while loops** continue until a condition becomes False

break exits loops immediately; **continue** skips to the next iteration

Lists are ordered and mutable - use for collections that change

Tuples are ordered and immutable - use for fixed records and function returns

Dictionaries store key-value pairs - use for fast lookups by meaningful keys

Sets contain unique elements only - use for membership testing and removing duplicates

Next Session Preview

Session 2.2: Functions & Modular Code

You will learn to:

- Define reusable functions with `def`
- Work with parameters and return values
- Understand variable scope (local vs global)
- Write docstrings for documentation
- Apply functions to build modular programs

Preparation: Review the Contact Book mini-project - we will refactor it using functions!

Vishlesan i-Hub IIT Patna x Masai School